Lab 10 – A Chatbot with Pydantic AI

Study-Room Desk Student Sheet 100 minutes

AI1220 — Software, Web & Mobile Engineering: Code without Limits

Goal

Build a chatbot for the library's study-room desk that you can talk to in the terminal: one Python file with one Pydantic AI agent. It runs first on a model on your laptop (LM Studio), then, with the same code, on IFM's hosted K2 Horizon model with your own API key.

Set up

1. Install LM Studio from https://lmstudio.ai and open it once. Download a GGUF chat model of at most 6 GB, for example `lms get qwen/qwen3-4b-2507@q4_k_m --gguf -y`.
2. Extract `lab10_files.zip`, rename the folder to `lab10` and move it into your `ai1220` repository. In `ai1220/lab10`, run `uv sync`, then `uv run pytest` (8 offline checks; all pass at the end).
3. Run `lms load MODEL --context-length 8192`, then `lms server start`. Put the identifier that `lms ps` shows after `LLM_MODEL=` in `lmstudio.env`.
4. Once Exercise 1 is done, chat with your model:

```
uv run --env-file lmstudio.env python chat.py
```

You write `chat.py`; apart from `lmstudio.env`, keep the other files as they are. An error that mentions a missing model means the identifier in `lmstudio.env` is wrong.

Pydantic AI in brief

Model	`OpenAIChatModel(name, provider=OpenAIProvider(base_url=..., api_key=...))` talks to any OpenAI-compatible server (`pydantic_ai.models.openai`, `pydantic_ai.providers.openai`).
Agent	`Agent(model, instructions=...)`. `result = agent.run_sync(text, message_history=..., model_settings=...)` runs one turn; `result.output` is the reply text.
Settings	`model_settings={"extra_headers": {"Name": "value"}}` in `agent.run_sync` adds an HTTP header to that run's model requests.
Messages	`result.all_messages()` is the whole conversation so far; pass it as `message_history` to the next turn.
Errors	A failed run raises a subclass of `pydantic_ai.exceptions.AgentRunError`.

The program

`chat.py` defines `agent` and a function `main()`, and ends with `if __name__ == "__main__": main()`. A session looks like this:

```
Study-room desk. Type /new for a new conversation and /quit to leave.
you> Hello!
desk> Hello! How can I help you with studying or the study rooms?
you> /quit
```

Exercise 1: Connect the agent to LM Studio 35%

In `chat.py`, create `agent` with an `OpenAIChatModel` whose name, base URL and key come from the environment variables `LLM_MODEL`, `LLM_BASE_URL` and `LLM_API_KEY`. Its instructions make it the assistant of the library's study-room desk that answers questions about studying and study rooms, in at most three sentences. `main()` prints the first line of the session above, then repeats: read a line with `input("you> ")`, skip it when it is empty, send it to the agent and print `desk>` and the reply. It ends on `/quit` and at the end of the input. When the model fails (`AgentRunError`), it prints the error and the chat goes on.

Exercise 2: Remember the conversation 30%

Keep the messages of the conversation and pass them to the next turn. Store them only after a successful turn. The line `/new` forgets them, prints `New conversation.` and starts again; `/new` and `/quit` never go to the model.

Exercise 3: Switch to IFM 35%

Register at https://platform.ifm.ai and create an API key. Copy `ifm.env.example` to `ifm.env` and add your key. The key goes only into `ifm.env`, which stays out of git; check `git status` before you commit. Send the header `X-Session-ID` on every model request, through `model_settings` (IFM session routing). Its value is a random id (for example `uuid.uuid4().hex`) that stays the same during a conversation and changes after `/new`. Chat with `--env-file ifm.env`, then run `check_chat.py` with `ifm.env` and the label `ifm`. K2 Horizon reasons before it answers, so a reply can take a minute. Show the TA an IFM answer before you leave. Marks: header 10, `evidence/ifm.json` 15 (only if the TA saw it), no key in your repository or its history 10.

Check your work

`uv run pytest` passes all 8 checks. Then, with your model running, chat these two messages:

1. `My name is Sam and I study physics.`
2. `What is my name?` The reply contains Sam, which needs the memory of Exercise 2.

Then run `uv run --env-file lmstudio.env python check_chat.py lmstudio`. It types a greeting and the same two messages into your `main()` and saves `evidence/lmstudio.json` when all three get an answer.

Submit

Push before the lab ends:

```
git add lab10 && git commit -m "Lab 10" && git push
```

Marks: Exercise 1 has 30 marks from checks and 5 from `evidence/lmstudio.json`; Exercise 2 has 30 marks from checks; Exercise 3 is listed above. The marking runs more checks like those of `uv run pytest`, with other model names, URLs and replies, and reads `evidence/lmstudio.json` and `evidence/ifm.json`. Each part earns the share of its checks that pass.